

Calculator documentation

Generated by Doxygen 1.13.2

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 calculator_GUI.py File Reference	3
2.1.1 Detailed Description	5
2.1.2 Function Documentation	5
2.1.2.1 format_number()	5
2.1.2.2 handle_abs()	5
2.1.2.3 handle_digit()	5
2.1.2.4 handle_equal()	6
2.1.2.5 handle_fact()	6
2.1.2.6 handle_operator()	6
2.1.2.7 keypress_handler()	7
2.1.2.8 on_enter()	8
2.1.2.9 show_error()	8
2.1.2.10 update_input()	8
2.1.3 Variable Documentation	9
2.1.3.1 btn	9
2.1.3.2 button_frame	9
2.1.3.3 buttons	9
2.1.3.4 current_input	9
2.1.3.5 display_frame	10
2.1.3.6 outer_frame	10
2.2 math_lib.py File Reference	10
2.2.1 Detailed Description	10
2.2.2 Function Documentation	11
2.2.2.1 absolute()	11
2.2.2.2 add()	11
2.2.2.3 div()	11
2.2.2.4 factorial()	12
2.2.2.5 modulo()	12
2.2.2.6 mult()	12
2.2.2.7 n_power()	13
2.2.2.8 n_root()	13
2.2.2.9 sub()	13
Index	15

Chapter 1

File Index

1.1 File List

Here is a list of all documented files with brief descriptions:

calculator_GUI.py	
GUI with history and error display	3
math_lib.py	
Library containing various simple math functions	10

Chapter 2

File Documentation

2.1 calculator_GUI.py File Reference

GUI with history and error display.

Functions

- `keypress_handler` (event)
Handle keyboard key presses.
- `format_number` (n)
Bind the keyboard input to the calculator.
- `show_error` (message)
Show error in popup message window.
- `reset_background` ()
Resets the window background to normal after error flash.
- `clear_all` ()
Clears all calculator state and resets display.
- `backspace` ()
Removes last character from current input.
- `update_input` (value)
Main handler for all button presses.
- `handle_digit` (digit)
Appends digit or sign to current input string.
- `handle_operator` (op)
Handles operator button press and stores operand/operator.
- `handle_equal` (chain=False)
Executes the stored operation and updates the result.
- `handle_abs` ()
Handles absolute value operation ($|x|$)
- `handle_fact` ()
Handles factorial operation ($!$)
- `on_enter` (e)
Add hover effect.
- `on_leave` (e)

Variables

- **script_dir** = sys._MEIPASS
Determine the correct path for assets depending on whether it's running from source or a bundled executable.
- str **current_input** = ""
Set working directory to the script's directory to ensure assets like logo.png load correctly.
- str **history_input** = "0"
- **current_operator** = None
- **operand** = None
- bool **just_evaluated** = False
- **window** = Tk()
GUI Window Setup.
- **icon** = PhotoImage(file='logo.png')
Set application icon using a PNG file Set application background colour.
- **background**
- **display_frame** = Frame(**window**, bg="#E79A3F")
Display Frame for history and current input.
- **fill**
- **padx**
- **pady**
- **outer_frame** = Frame(**window**, bg="#E79A3F")
Outer frame to modify the height of the vertical height of the buttons.
- **expand**
- **False**
- **button_frame** = Frame(**outer_frame**, bg="#E79A3F", height=200)
Frame to contain the button grid.
- **history_label** = Label(**display_frame**, **text**=history_input, anchor="e", font=("Arial", 16), bg="#E79A3F", fg="#444")
Label showing the previously entered values or result history Located on the top of the display_frame.
- **input_label** = Label(**display_frame**, **text**="0", anchor="e", font=("Arial", 32), bg="#E79A3F", fg="#000")
Label showing the current number input or result Located on the bottom of the display_frame.
- list **buttons**
Calculator Button Grid Layout.
- **True**
- **i**
Configure the rows and columns of the button grid to expand equally.
- **weight**
Configure the rows and columns of the button grid to expand equally.
- **uniform**
- **btn**
Create and place calculator buttons.
- **row**
- **row_index**
- **column**
- **col_index**
- **sticky**
- **text**
Initialize the label displays.

2.1.1 Detailed Description

GUI with history and error display.

Author

Šimon Urban

This module provides basic user interface for the calculator

2.1.2 Function Documentation

2.1.2.1 format_number()

```
format_number (
    n)
```

Bind the keyboard input to the calculator.

Converts float to int if it's a whole number

Parameters

<i>n</i>	The number to format
----------	----------------------

Returns

Integer if whole number, otherwise rounded float

2.1.2.2 handle_abs()

```
handle_abs ()
```

Handles absolute value operation ($|x|$)

Returns

Updates input label with absolute value

Calls [math_lib.absolute\(\)](#) and replaces current_input with the result. Used for $|x|$ button.

2.1.2.3 handle_digit()

```
handle_digit (
    digit)
```

Appends digit or sign to current input string.

Parameters

<i>digit</i>	The string representing the digit (0–9) or negative sign ('-')
--------------	--

Builds up the number the user is typing. Called each time the user clicks a digit. Allow toggling minus sign at the beginning

2.1.2.4 handle_equal()

```
handle_equal (
    chain = False)
```

Executes the stored operation and updates the result.

Parameters

<i>chain</i>	True if part of a chained operation
--------------	-------------------------------------

Returns

Displays result or error message

Based on the stored operator and the current input, this function performs the corresponding math operation using `math_lib`. It formats the result and updates both the history and input labels accordingly. It also handles chaining if another operator follows right after evaluation.

2.1.2.5 handle_fact()

```
handle_fact ()
```

Handles factorial operation (!)

Returns

Updates input label with factorial result

Converts `current_input` to int and uses `math_lib.factorial()`. Shows error if the number is negative or not an integer.

2.1.2.6 handle_operator()

```
handle_operator (
    op)
```

Handles operator button press and stores operand/operator.

Parameters

<i>op</i>	Operator character pressed
-----------	----------------------------

Evaluates any existing operation if operand and operator are already set, then stores result as new operand and prepares for next input. Also supports using '-' to type negative numbers. If the user is entering a negative number, allow '-' to be part of input. If '-' pressed and typing a new negative number. If operator already set but no second number typed, replace operator

2.1.2.7 keypress_handler()

```
keypress_handler (  
    event)
```

Handle keyboard key presses.

Parameters

<i>event</i>	The event object from tkinter
--------------	-------------------------------

This maps keyboard keys to calculator input

2.1.2.8 on_enter()

```
on_enter (  
    e)
```

Add hover effect.

Parameters

<i>event</i>	object (button)
--------------	-----------------

2.1.2.9 show_error()

```
show_error (  
    message)
```

Show error in popup message window.

Parameters

<i>message</i>	Error message to display
----------------	--------------------------

Show error in popup message window and flash whole window background red

Parameters

<i>message</i>	Error message to display
----------------	--------------------------

2.1.2.10 update_input()

```
update_input (  
    value)
```

Main handler for all button presses.

Parameters

<i>value</i>	The label of the button pressed
--------------	---------------------------------

This function routes each button press to the appropriate handler depending on the type:

- Digits are added to the input string
- Operators are saved for later evaluation
- '=' triggers computation
- Utility buttons like 'C', '|x|' and '!' trigger their own logic If the last action was '=', pressing any new button clears the state to start a fresh calculation.

2.1.3 Variable Documentation

2.1.3.1 btn

btn

Initial value:

```
00001 = Button(
00002     button_frame,
00003     text=label,
00004     font=("Arial", 18),
00005     bg="#F9E0AE",
00006     activebackground="#F0C674",
00007     relief=RAISED,
00008     bd=3,
00009     command=lambda val=label: update_input(val)
00010 )
```

Create and place calculator buttons.

2.1.3.2 button_frame

```
button_frame = Frame(outer_frame, bg="#E79A3F", height=200)
```

Frame to contain the button grid.

Frame to contain the button grid Pack buttons so they fill out the frame.

2.1.3.3 buttons

list buttons

Initial value:

```
00001 = [
00002     ["C", "", "%", "", ""],
00003     ["7", "8", "9", "^", ""],
00004     ["4", "5", "6", "/", "-"],
00005     ["1", "2", "3", "*", "+"],
00006     ["0", ".", "!", "|x|", "="]
00007 ]
```

Calculator Button Grid Layout.

Rows represent groups of calculator buttons

2.1.3.4 current_input

```
str current_input = ""
```

Set working directory to the script's directory to ensure assets like logo.png load correctly.

Calculator State Variables

These hold calculator state between user actions

2.1.3.5 display_frame

```
display_frame = Frame(window, bg="#E79A3F")
```

Display Frame for history and current input.

This section contains two labels stacked vertically

2.1.3.6 outer_frame

```
outer_frame = Frame(window, bg="#E79A3F")
```

Outer frame to modify the height of the vertical height of the buttons.

Frame to limit button area

2.2 math_lib.py File Reference

Library containing various simple math functions.

Functions

- `add` (number1, number2)
Adds two numbers.
- `sub` (number1, number2)
Subtracts two numbers.
- `div` (number1, number2)
Divides two numbers.
- `mult` (number1, number2)
Multiplies two numbers.
- `absolute` (number)
Finds absolute value of a given number.
- `factorial` (number)
Calculates factorial of a number.
- `modulo` (number1, number2)
Calculates remainder after division.
- `n_power` (number1, number2)
Calculates the natural power of number.
- `n_root` (number1, number2)
Calculates the natural root of number.

2.2.1 Detailed Description

Library containing various simple math functions.

Author

Martin Mezei

Adam Ivan

This module provides basic arithmetic operations

2.2.2 Function Documentation

2.2.2.1 absolute()

```
absolute (  
    number)
```

Finds absolute value of a given number.

Parameters

<i>number</i>	Argument of the absolute value function
---------------	---

Returns

Absolute value of a number

2.2.2.2 add()

```
add (  
    number1,  
    number2)
```

Adds two numbers.

Function add() takes two numbers as input and adds them

Parameters

<i>number1</i>	First number of addition
<i>number2</i>	Second number of addition

Returns

Sum of number1 and number2

2.2.2.3 div()

```
div (  
    number1,  
    number2)
```

Divides two numbers.

Function div() takes two numbers as input and divides them If zero division is attempted ZeroDivisionError is raised

Parameters

<i>number1</i>	Dividend
<i>number2</i>	Divisor

Returns

Division quotient

2.2.2.4 factorial()

```
factorial (  
    number)
```

Calculates factorial of a number.

Function factorial() takes one number which is either 0 or a natural number and calculates its factorial. If a calculation with a number that is not 0 or natural is attempted, ValueError is raised.

Parameters

<i>number</i>	Argument of factorial function
---------------	--------------------------------

Returns

Value of factorial for the given number

2.2.2.5 modulo()

```
modulo (  
    number1,  
    number2)
```

Calculates remainder after division.

Function modulo() takes two numbers and returns the remainder after division. As in the div() function, returns ZeroDivisionError if using modulo with 0.

Parameters

<i>number1</i>	First number in division
<i>number2</i>	Second number in division

Returns

Remainder after division

2.2.2.6 mult()

```
mult (  
    number1,  
    number2)
```

Multiplies two numbers.

Function mult() takes two numbers as input and multiplies them.

Parameters

<i>number1</i>	First number of multiplication
<i>number2</i>	Second number of multiplication

Returns

Multiplication product

2.2.2.7 n_power()

```
n_power (
    number1,
    number2)
```

Calculates the natural power of number.

Function `n_power()` takes two numbers and calculates `number1` to the power of `number2`. This function raises `ValueError` if not using natural numbers in exponents.

Parameters

<i>number1</i>	Base of exponentiation
<i>number2</i>	Exponent of exponentiation

Returns

Value of `nth` power for the given numbers

2.2.2.8 n_root()

```
n_root (
    number1,
    number2)
```

Calculates the natural root of number.

Function `n_root()` takes two numbers and calculates the `number2` root of `number1`. This function raises `ValueError` if not using natural numbers in the exponents of root. Allows negative radicands only if the root degree is odd.

Parameters

<i>number1</i>	Radicand of root
<i>number2</i>	Degree of root

Returns

Value of `nth` root for the given numbers

2.2.2.9 sub()

```
sub (
    number1,
    number2)
```

Subtracts two numbers.

Function `sub()` takes two numbers as input and subtracts them.

Parameters

<i>number1</i>	First number of subtraction
<i>number2</i>	Second number of subtraction

Returns

Difference of number1 and number2

Index

- absolute
 - math_lib.py, [11](#)
- add
 - math_lib.py, [11](#)
- btn
 - calculator_GUI.py, [9](#)
- button_frame
 - calculator_GUI.py, [9](#)
- buttons
 - calculator_GUI.py, [9](#)
- calculator_GUI.py, [3](#)
 - btn, [9](#)
 - button_frame, [9](#)
 - buttons, [9](#)
 - current_input, [9](#)
 - display_frame, [9](#)
 - format_number, [5](#)
 - handle_abs, [5](#)
 - handle_digit, [5](#)
 - handle_equal, [6](#)
 - handle_fact, [6](#)
 - handle_operator, [6](#)
 - keypress_handler, [6](#)
 - on_enter, [8](#)
 - outer_frame, [10](#)
 - show_error, [8](#)
 - update_input, [8](#)
- current_input
 - calculator_GUI.py, [9](#)
- display_frame
 - calculator_GUI.py, [9](#)
- div
 - math_lib.py, [11](#)
- factorial
 - math_lib.py, [11](#)
- format_number
 - calculator_GUI.py, [5](#)
- handle_abs
 - calculator_GUI.py, [5](#)
- handle_digit
 - calculator_GUI.py, [5](#)
- handle_equal
 - calculator_GUI.py, [6](#)
- handle_fact
 - calculator_GUI.py, [6](#)
- handle_operator
 - calculator_GUI.py, [6](#)
- calculator_GUI.py, [6](#)
- keypress_handler
 - calculator_GUI.py, [6](#)
- math_lib.py, [10](#)
 - absolute, [11](#)
 - add, [11](#)
 - div, [11](#)
 - factorial, [11](#)
 - modulo, [12](#)
 - mult, [12](#)
 - n_power, [13](#)
 - n_root, [13](#)
 - sub, [13](#)
- modulo
 - math_lib.py, [12](#)
- mult
 - math_lib.py, [12](#)
- n_power
 - math_lib.py, [13](#)
- n_root
 - math_lib.py, [13](#)
- on_enter
 - calculator_GUI.py, [8](#)
- outer_frame
 - calculator_GUI.py, [10](#)
- show_error
 - calculator_GUI.py, [8](#)
- sub
 - math_lib.py, [13](#)
- update_input
 - calculator_GUI.py, [8](#)